Let’s build an LLM agent 🤖 in Pyton 🐍

Step-by-step

→

Why agents?

Because Large Language Models alone are **not enough** to accurately answer complex tasks that require

> **External information** that was not present in the training dataset used to fit the LLM paramaters

or

> Many **reasoning steps**

For example

If you ask OpenAI gpt-3.5-turbo LLM to

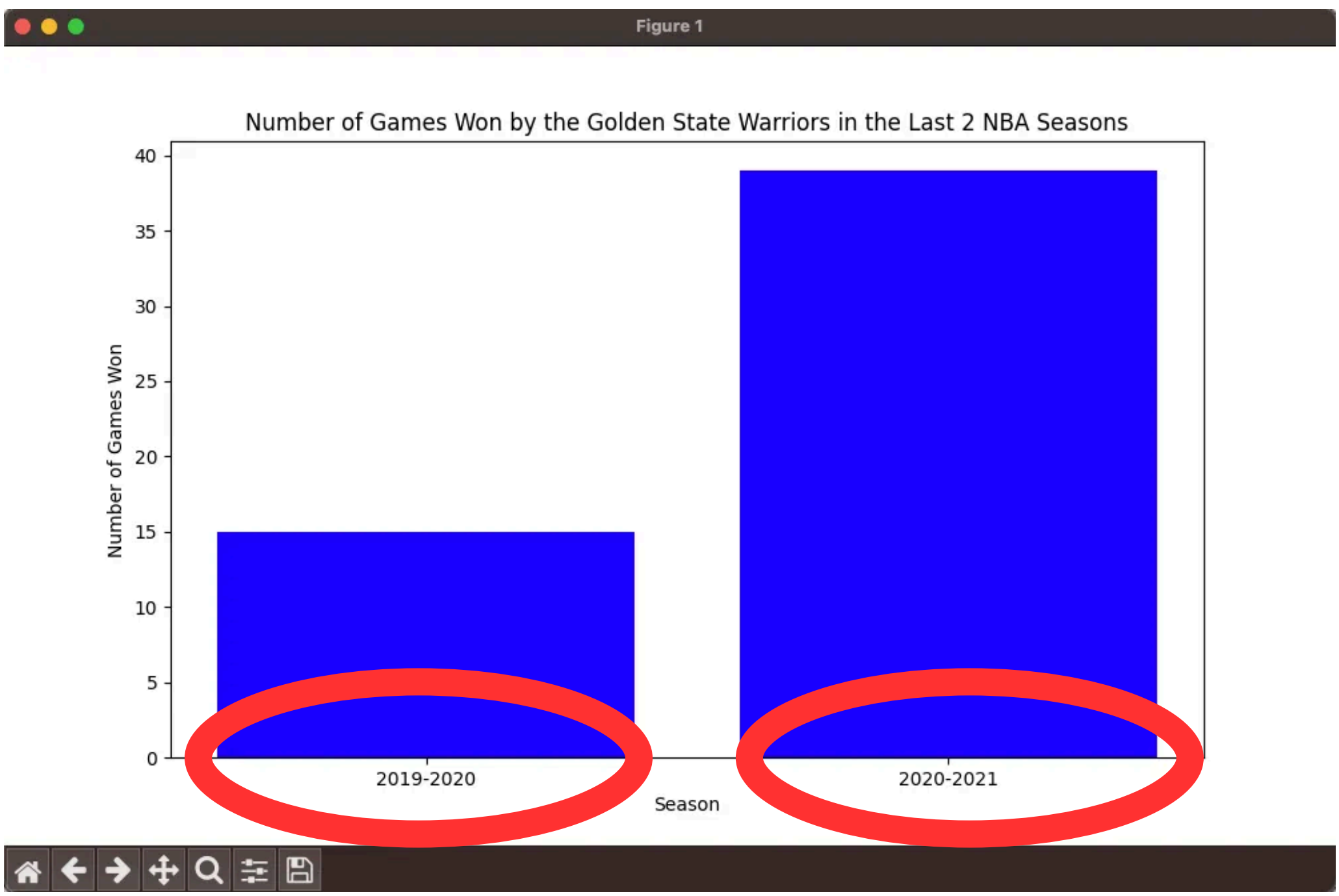
"***Create a plot with Python of the number of games won by the Golden State Warriors in each of the last 2 NBA seasons***."

The response you get is

-> **a 100% correct Python code** because GPT-3.5 was trained on vast amounts of Python code available on the Internet, and hence can work as a great code generator tool.

BUT

→ BUT, the data plotted is **wrong**, as it is not from the last 2 seasons, but from 2019 and 2020.



How to fix this?

To fix this you need to supercharge your LLM with

→ **A tool** to retrieve external data needed for the task

and

→ The **reasoning capabilities** to decide when and how to use the tool, to answer the user query.

And this is what **agents** 🤖 are all about.

Let’s build one step-by-step →

Step 1. Pick your LLM

You can either use

→ a local LLM running on your laptop with Ollama, or

→ connect to an external API like OpenAI or Cohere.

```python
def get_chat_model(model_provider: str, model_name: str) -> BaseChatModel:
    """
    Return a chat-based LLM model from the given `model_provider` with the given `model_name`.

    Args:
        model_provider (str): The provider of the chat-based LLM model.
        model_name (str): The name of the chat-based LLM model to use.

    Returns:
        ChatOllama: A chat-based LLM model.
    """
    assert model_provider in [
        'ollama',
        'cohere',
        'openai'
    ], f'Invalid model provider: {model_provider}'

    if model_provider == 'ollama':
        logger.info(f'Loading Ollama model: {model_name}')
        llm = ChatOllama(model=model_name, temperature=0.0)

    elif model_provider == 'cohere':
        logger.info(f'Loading Cohere model: {model_name}')
        llm = ChatCohere(model=model_name, temperature=0.0)

    elif model_provider == 'openai':
        logger.info(f'Loading OpenAI model: {model_name}')
        llm = ChatOpenAI(model=model_name, temperature=0.0)

    return llm
```

Step 2. Define your tools 🛠️

In this case we need:

1) An **Internet search** tool

```
def get_tabily_search_tool() -> Tool:
    """
    Returns a tool that searches the internet for a query using the Tavily API.

    Returns:
        TavilySearchResults: A tool that searches the internet for a query using the Tavily API.
    """
    internet_search = TavilySearchResults()
    internet_search.name = 'internet_search'
    internet_search.description = 'Returns a list of relevant document snippets for a textual query retrieved
from the internet.'

    class TavilySearchInput(BaseModel):
        query: str = Field(description='Query to search the internet with')

    internet_search.args_schema = TavilySearchInput

    return internet_search
```

2) A **PythonREPL** so the Agent can run Python code to debug its output

```python
def get_python_interpreter_tool() -> Tool:
    """
    Creates a tool that executes python code and returns the result.

    Returns:
        Tool: A tool that executes python code and returns the result.
    """
    python_repl = PythonREPL()

    repl_tool = Tool(
        name='python_repl',
        description='Executes python code and returns the result. The code runs in a
static sandbox without interactive mode, so print output or save output to a file.',
        func=python_repl.run,
    )
    repl_tool.name = 'python_interpreter'

    # from langchain_core.pydantic_v1 import BaseModel, Field
    class ToolInput(BaseModel):
        code: str = Field(description='Python code to execute.')

    repl_tool.args_schema = ToolInput

    return repl_tool
```

Step 3. Build the ReAct agent

This step is super-simple thanks to a library like LangChainA, which provides an implementation of the ReAct agent logic.

We first define the agent logic, using the

> **tools** we created
> the base **LLM** we picked, and
> an **initial prompt** of that can help us steer the the agent in the right direction

```
agent = create_react_agent(
    llm=llm,
    tools=tools,
    prompt=prompt,
)
```

From this agent logic, we define the **AgentExecutor**, which is the runtime for an agent, that

> **calls** the agent
> **executes** the actions it chooses
> passes the action outputs **back to the agent**
> and **repeats**

```
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True

)
```

Step 4. Run the agent 🏃

We pass the input query

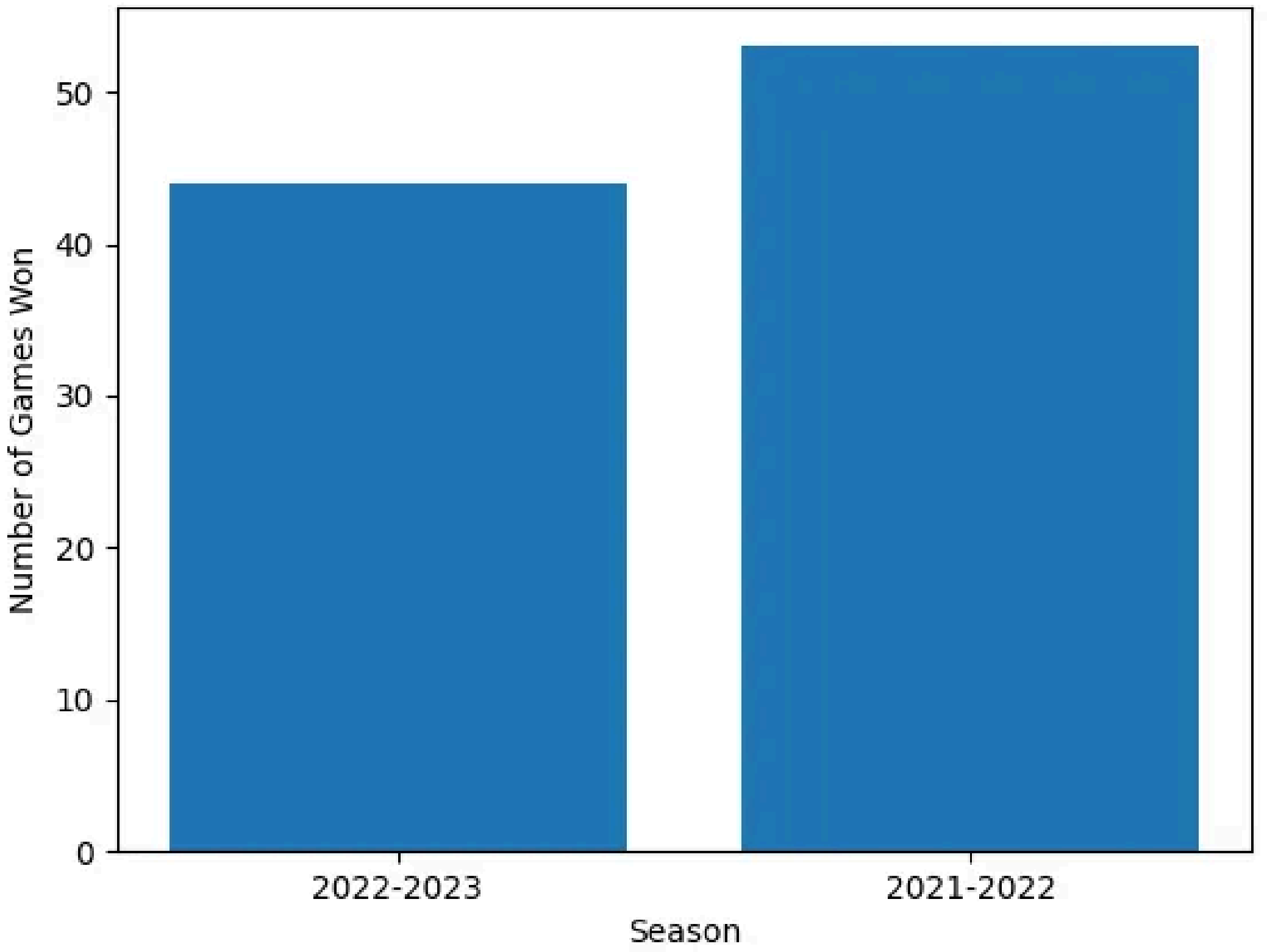
"***Create a plot with Python of the number of games won by the Golden State Warriors in each of the last 2 NBA seasons***."

to the agent_executor

```
agent_executor.invoke({'input': input})
```

...and the agent start reasoning and acting.

Until it produces the **correct plot**!



Attention 📢

As you start experimenting with different input prompts, you will find scenarios in which the agent is not able to solve the task.

Moving from agent prototypes to production-ready agents is all about **polishing rough edges**, and ensuring the agent has access to the right tools.

But this is something we will leave for another day.

Wanna learn to build LLM apps?

Every day I share free, hands-on content, on production-grade ML, to help you build real-world ML products.

Follow me and **click on the** 🔔 so you don't miss what's coming next